Dependency Inversion Principle (DIP) - SOLID in Java

1. What is DIP?

The Dependency Inversion Principle is the "D" in SOLID.

Rule 1: High-level modules should NOT depend on low-level modules.
Both should depend on ABSTRACTIONS.

Rule 2: Abstractions should NOT depend on details.
Details should depend on ABSTRACTIONS.

In plain English: Program to interfaces, not to implementations.

2. The Problem Without DIP

Use Case: Order Processing System

OrderService directly instantiates EmailNotifier -- tightly coupled.

```
// Low-level module
public class EmailNotifier {
    public void sendEmail(String message) {
        System.out.println("Sending EMAIL: " + message);
    }
}

// High-level module -- DIRECTLY depends on low-level (WRONG)
public class OrderService {
    private EmailNotifier notifier = new EmailNotifier(); // hard dependency!

    public void processOrder(Order order) {
        // ... process logic
        notifier.sendEmail("Order placed: " + order.getId());
    }
}
```

Problems with this approach

Problem	Impact
OrderService hardwired to EmailNotifier	Cannot switch to SMS/Push without changing OrderService
Tight coupling	Unit testing requires a real EmailNotifier
Violates Open/Closed too	Every new notification type needs core service changes

Dependency Inversion Principle (DIP) - SOLID in Java

3. The Fix -- Apply DIP

Step 1: Define an abstraction (interface)

```
// The abstraction -- both high & low level modules depend on THIS
public interface NotificationService {
    void notify(String message);
}
```

Step 2: Low-level modules implement the abstraction

```
public class EmailNotifier implements NotificationService {
    @Override
    public void notify(String message) {
        System.out.println("EMAIL: " + message);
    }
}

public class SMSNotifier implements NotificationService {
    @Override
    public void notify(String message) {
        System.out.println("SMS: " + message);
    }
}

public class PushNotifier implements NotificationService {
    @Override
    public void notify(String message) {
        System.out.println("PUSH: " + message);
    }
}
```

Step 3: High-level module depends on the abstraction

```
public class OrderService {
    private final NotificationService notifier; // interface, not concrete!

    // Dependency INJECTED from outside (constructor injection)
    public OrderService(NotificationService notifier) {
        this.notifier = notifier;
    }

    public void processOrder(Order order) {
        // ... business logic
        notifier.notify("Order placed: " + order.getId());
    }
}
```

Step 4: Wiring -- Spring Boot or plain main()

```
// Spring Boot -- Spring resolves the correct bean automatically
@Service
public class OrderService {
    private final NotificationService notifier;
```

Dependency Inversion Principle (DIP) - SOLID in Java

```
    @Autowired
    public OrderService(NotificationService notifier) {
        this.notifier = notifier;
    }
}

@Component
public class EmailNotifier implements NotificationService { ... }

// -- OR -- plain main()
public class Main {
    public static void main(String[] args) {
        NotificationService notifier = new SMSNotifier(); // swap anytime!
        OrderService service = new OrderService(notifier);
        service.processOrder(new Order("ORD-101"));
    }
}
```

Dependency Inversion Principle (DIP) - SOLID in Java

4. Unit Testing Becomes Trivial

```
@Test
public void testOrderService() {
    // Mock the abstraction -- no real email/SMS sent!
    NotificationService mockNotifier = Mockito.mock(NotificationService.class);
    OrderService service = new OrderService(mockNotifier);

    service.processOrder(new Order("ORD-001"));

    Mockito.verify(mockNotifier).notify("Order placed: ORD-001");
}
```

5. Before vs After Comparison

Before DIP	After DIP
OrderService creates new EmailNotifier()	Dependency injected via constructor
Hardwired to one notification type	Can swap Email/SMS/Push at runtime
Hard to unit test	Easily mockable with Mockito
Adding new notifier = modifying core service	Just add a new class implementing the interface

6. Key Insight

Think of a POWER SOCKET (abstraction). Your laptop, phone, and lamp all plug into the same socket without the wall caring what is plugged in.
The socket = the interface. Devices = the implementations.

This is also the foundation of how Spring's IoC container and dependency injection work under the hood.